

GT-Bench: Retention-Aware Fine-Tuning for Verifiable Game-Theory Reasoning

Mert Gulsum
University of California, Berkeley
`mert_gulsum@berkeley.edu`

May 12, 2026

Abstract

GT-Bench is a compact benchmark for measuring whether targeted fine-tuning improves exactly verifiable game-theory reasoning. It has two layers: a canonical 2x2 pure-equilibrium task and a broader six-family suite covering mixed 2x2 equilibria, dominance, larger normal-form games, extensive-form games, natural-language game descriptions, and repeated-interaction questions. Each canonical example gives a randomly generated 2x2 two-player normal-form payoff matrix and asks a model to identify all pure-strategy Nash equilibria. Because the tasks have exact solvers or exact answer metadata, data can be generated at scale and model outputs can be scored without subjective judgment. Using Thinking Machines Lab Tinker, LoRA supervised fine-tuning of `Qwen/Qwen3.6-27B` on 5000 synthetic examples improved canonical exact-match accuracy from 87.60% to 99.60%. A conservative 500-example adversarial prompt supplement further improved prompt-robustness accuracy from 88.40% to 98.80% relative to the original 5000-example checkpoint, without reducing canonical accuracy. A retention-aware multitask sweep then selected a joint checkpoint that reached 91.67% on the broader suite while retaining 99.80% canonical accuracy and 99.60% prompt-robustness accuracy. Repeating the selected recipe across three seeds gave 92.56% mean suite accuracy and 99.93% mean canonical accuracy. The result supports a bounded claim: targeted fine-tuning can improve one controlled formal reasoning skill, and retention-aware multitask training data can broaden exact game-theory coverage without erasing that skill. It does not demonstrate general game-theory competence.

1 Introduction

Large language models often appear capable of formal reasoning, but broad qualitative examples make it difficult to separate rule acquisition from prompt familiarity, memorized patterns, or forgiving evaluation. GT-Bench is intentionally synthetic, mechanically verifiable, and bounded. The canonical experiment isolates one small mathematical task: given a 2x2 normal-form game, return exactly the set of pure-strategy Nash equilibria. The broader suite then asks whether that exact-answer coverage can expand to mixed equilibria, dominance, larger normal-form games, extensive-form games, natural-language descriptions, and repeated-interaction questions without forgetting the canonical skill. The artifact is a benchmark for controlled game-theory reasoning tasks, not evidence of broad game-theory competence.

The experimental question is practical: can targeted supervised fine-tuning measurably improve a model on precisely specified formal game-theory skills? The project was developed as a public research artifact for a Thinking Machines Lab Tinker research grant. The design goal is control:

exact solvers or exact answer metadata, deterministic data generation, exact-match scoring, and focused robustness checks.

The main result is positive. On the canonical 500-example held-out test split, `Qwen/Qwen3.6-27B` improved from 87.60% exact-match accuracy at baseline to 99.60% after LoRA supervised fine-tuning on 5000 synthetic examples. The improvement held on an independent confirmation set, became sharper on a balanced stress set, and remained stable in a three-seed training-data sweep. A follow-up adversarial prompt supplement closed most of the remaining prompt-format robustness gap. The repository also includes an exactly scored broader suite. A suite-only checkpoint improved suite accuracy but forgot the canonical task; the final selected joint checkpoint preserves the canonical result while raising suite accuracy above 90%.

2 Related Work

GT-Bench sits at the intersection of formal reasoning evaluation, synthetic verifiable tasks, parameter-efficient fine-tuning, and LLM game-theory evaluation. Mathematical reasoning benchmarks such as `GSM8K` and `MATH` evaluate models on natural-language math problems with exact or semi-exact final answers [2, 4]. Code benchmarks such as `HumanEval` and `LiveCodeBench` use executable tests or live contamination-limited problem streams to reduce subjective grading and benchmark leakage [1, 6]. Recent arithmetic benchmark work also emphasizes the need to distinguish generalization from memorization when benchmark items may appear in pretraining data [13]. GT-Bench follows this verifiable-evaluation direction but uses a fully synthetic, regenerable task with an exact solver.

The fine-tuning intervention uses LoRA [5], a parameter-efficient adaptation method. The base model is `Qwen/Qwen3.6-27B`, a 27B open-weight Qwen3.6 model distributed by the Qwen team under Apache 2.0 [9]; its broader model family is described in the Qwen3 technical report [12]. Training was run with Tinker, a managed fine-tuning API that exposes LoRA training and sampling while handling distributed infrastructure [11].

There is also a growing body of work on LLMs and strategic reasoning. Surveys frame this area as including game-based evaluation, game-theoretic alignment, and LLM-assisted game analysis [10]. Existing game-theory-oriented LLM benchmarks such as `GTBench` evaluate broader strategic behavior in game environments [3]. This paper is deliberately more controlled: it does not evaluate open-ended play, negotiation, or strategic behavior in interactive environments. It tests whether a model can learn exact game-theory reasoning tasks, starting with one canonical 2x2 pure-equilibrium rule and then broadening to a fixed six-family exact-answer suite.

3 Canonical Task Definition

Each example is a two-player normal-form game. Player 1 chooses U or D , and Player 2 chooses L or R . Each cell contains an ordered pair of integer payoffs in the range 0 through 9. A pure-strategy Nash equilibrium is a strategy profile in which both players are best responding to the other player’s strategy [7, 8]. Ties are handled exactly, so a game may have zero, one, two, three, or four pure equilibria.

The canonical model experiment deliberately excludes mixed strategies, dominance, welfare analysis, sequential games, auctions, public goods games, Nim, and natural-language story problems. This restriction is central to the original result. The broader suite adds several of these task families, but still evaluates fixed exact-answer tasks rather than open-ended game-theory competence.

4 Broader Suite Infrastructure

The public repository now also contains a broader suite generator and scorer. This suite covers six additional task families: fully mixed-equilibrium computation for 2x2 games with no pure equilibrium, iterated elimination of strictly dominated pure strategies, pure equilibria in 2x3 and 3x3 normal-form games, backward induction in perfect-information extensive-form games, natural-language game descriptions, and finite repeated prisoner’s-dilemma simulations plus best-response selection among fixed policies. Each family has deterministic generation, exact solver metadata, difficulty buckets, train/validation/test split generation, task-family scoring, local smoke baselines, and an oracle check.

The suite test split contains 300 examples, 50 from each family. Table 1 reports selected Tinker model-evaluation rows on this fixed split. The suite-only SFT checkpoint improved suite accuracy but retained only 53.60% on the canonical 2x2 task. The selected retention-aware joint checkpoint preserves canonical accuracy while improving the broader suite.

Table 1: Broader-suite accuracy by task family. The selected joint SFT row is the retention-aware checkpoint chosen by the rule in Section 12; the base-full-targeted row reaches higher suite accuracy but misses the retention gate.

Checkpoint	Overall	Mixed	Dominance	Large NF	Extensive	NL	Repeated
Base model	20.00%	0.00%	0.00%	16.00%	36.00%	26.00%	42.00%
2x2 SFT transfer	48.33%	0.00%	32.00%	72.00%	66.00%	54.00%	66.00%
2x2 prompt-adversarial transfer	51.67%	0.00%	32.00%	70.00%	80.00%	60.00%	68.00%
Suite SFT	68.00%	78.00%	44.00%	54.00%	76.00%	56.00%	100.00%
Selected joint SFT	91.67%	96.00%	68.00%	86.00%	100.00%	100.00%	100.00%
Joint base full targeted	93.00%	98.00%	78.00%	84.00%	98.00%	100.00%	100.00%

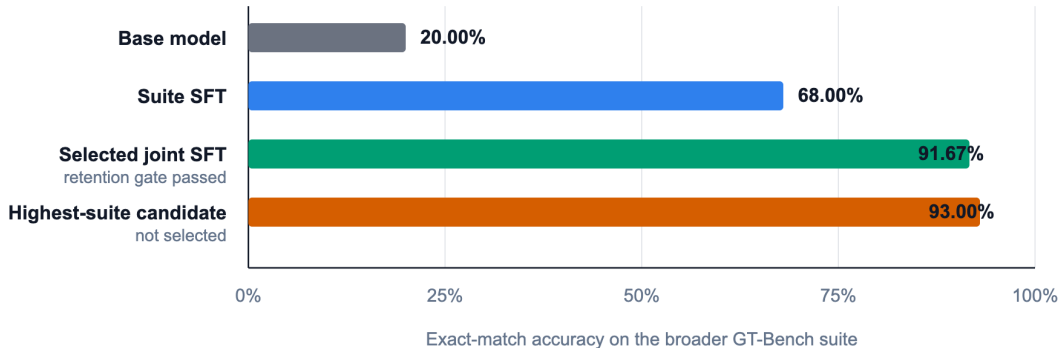


Figure 1: Overall broader-suite accuracy for the base model, the suite-only checkpoint, the selected joint checkpoint, and the highest-suite multitask candidate.

5 Data, Leakage Controls, and Scoring

The canonical dataset uses random seed 42 and contains 5000 training, 500 validation, and 500 held-out test examples. The generator samples eight integer payoffs from 0 through 9, rejects duplicate payoff matrices, and slices one jointly generated sequence into train, validation, and test splits. This makes the canonical train, validation, and test matrices disjoint by construction.

Additional evaluation sets were generated from separate seeds: a 1000-example confirmation set, a 250-example balanced stress set, and a 250-example prompt-robustness set. A signature audit over payoff matrices found no overlap between the canonical training split and the validation, canonical test, confirmation, stress, robustness, or adversarial supplement files. The adversarial supplement also avoids canonical training matrices by construction. It has one payoff-matrix overlap with the balanced stress set; the stress result is therefore reported mainly as a sanity check for the adversarial run, not as its primary evidence. The prompt-robustness result is the central adversarial follow-up.

Table 2: Dataset sizes and equilibrium-count distributions. “0 eq.” means no pure-strategy Nash equilibrium.

Split	Total	0 eq.	1 eq.	2 eq.	3 eq.	4 eq.
Train	5000	412	3234	1287	65	2
Validation	500	38	326	128	8	0
Canonical test	500	45	308	141	6	0
Confirmation	1000	94	642	255	9	0
Stress	250	50	50	50	50	50
Robustness	250	50	50	50	50	50

The exact solver computes each player’s best responses to each opponent action, then returns exactly those cells in which both players are best responding. The scorer parses common answer forms such as (U, L), U,L, and natural-language empty-set statements such as **none** or **no pure Nash equilibrium**. Predictions are converted into sets and compared to the solver output while ignoring order. Accuracy is exact-match accuracy: a prediction is correct only if it names exactly the gold equilibrium set. Unparseable predictions are scored incorrect. In the audited canonical baseline, canonical 5000-example SFT, adversarial canonical, and adversarial robustness prediction files, the parser found a prediction set for every example.

6 Fine-Tuning Setup and Baselines

The base model was Qwen/Qwen3.6-27B. Experiments used the Tinker `qwen3_disable_thinking` renderer and LoRA supervised fine-tuning [5] with rank 16, 3 epochs, effective batch size 32, maximum length 4096, and learning rate 10^{-4} . Decoding used temperature 0, one sample, and a 768-token maximum. Three canonical training sizes were evaluated: 250, 1000, and 5000 examples.

The local run manifests record 21 training steps for the 250-example runs, 93 steps for the 1000-example runs, 468 steps for the 5000-example runs, and 513 steps for the 5500-example adversarial run. Observed wall-clock training time was approximately 1–2 minutes for 250 examples, 6–7 minutes for 1000 examples, 31–32 minutes for 5000 examples, and 34 minutes for the adversarial 5500-example run. Exact token-billing and credit consumption were not committed to the public artifact; raw run manifests are intentionally ignored because they contain private Tinker sampler and state paths.

The main model baseline is the unmodified base model evaluated with the same deterministic decoding. Simple non-learned baselines are much weaker on the canonical test set: always predicting no pure equilibrium gets 45/500 examples correct (9.00%); always predicting the most frequent single equilibrium set, either (U, L) or (D, R) on this split, gets 83/500 correct (16.60%); uniformly guessing one of the 16 possible profile subsets has expected exact-match accuracy 6.25%. The exact solver is used only to generate labels and score predictions, not as a learned baseline.

7 Canonical Results

Table 3 shows exact-match accuracy on the 500-example canonical held-out test set. With 250 examples, performance fell; with 1000 examples, performance improved modestly; and with 5000 examples, the run nearly solved the split.

Table 3: Canonical exact-match accuracy on 500 held-out examples. Confidence intervals are exact binomial intervals.

Run	Accuracy	95% CI	Correct	Incorrect	Delta
Base model	87.60%	84.39–90.36%	438	62	0.00 pp
250-example SFT	53.40%	48.92–57.84%	267	233	-34.20 pp
1000-example SFT	91.60%	88.82–93.88%	458	42	+4.00 pp
5000-example SFT	99.60%	98.56–99.95%	498	2	+12.00 pp

The negative 250-example result is informative. Too little narrow training data can distort the learned behavior, even when the task is simple and exactly scored. The 1000-example run improved over baseline but remained far below the 5000-example condition.

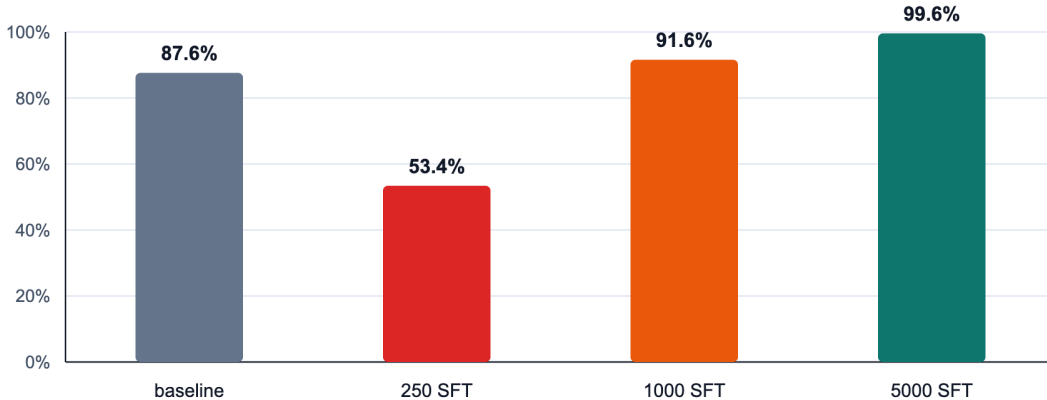


Figure 2: Canonical exact-match accuracy for the base model and three SFT training sizes.

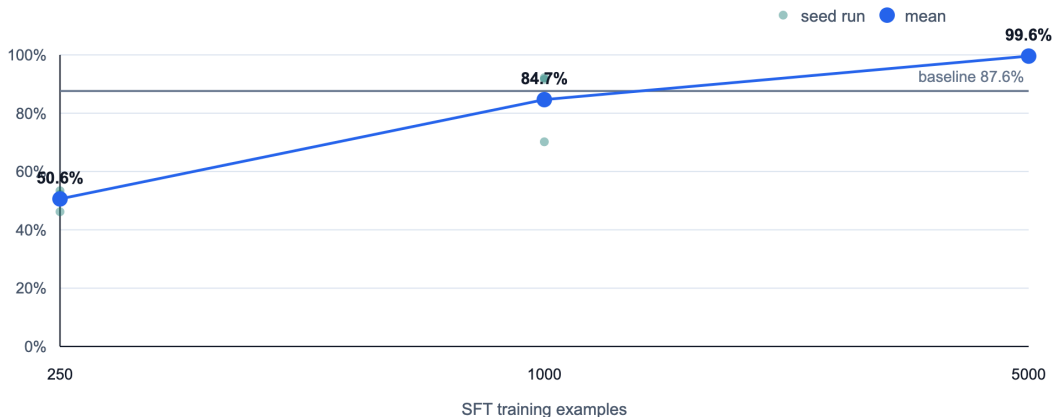
8 Repeated-Seed Learning Curve

To test whether the learning curve was sensitive to the synthetic training-data seed, the SFT sweep was repeated across three independent training-data seeds while keeping the canonical 500-example test set fixed. Seed 42 is the original sweep; seeds 1009 and 2027 were generated for this comparison.

The 5000-example condition was stable across all three seeds, with individual accuracies of 99.60%, 99.20%, and 100.00%. The 1000-example condition was volatile: two seeds improved over baseline, but seed 2027 fell to 70.20%. Because this is a three-seed comparison, the seed-level statistics should be read descriptively rather than as a strong hypothesis test.

Table 4: Repeated-seed learning curve on the fixed canonical test set.

Train size	Mean accuracy	Seed SD	Mean delta vs. baseline
250	50.60%	3.86 pp	-37.00 pp
1000	84.67%	12.53 pp	-2.93 pp
5000	99.60%	0.40 pp	+12.00 pp

**Figure 3:** Accuracy across three training-data seeds for each training size. The 5000-example condition is stable; the 1000-example condition is seed-sensitive.

9 Confirmation and Stress Evaluation

To check whether the result was specific to the canonical split, a fresh 1000-example confirmation set was generated with seed 20260505. The baseline achieved 89.20%, while the 5000-example checkpoint achieved 99.70%.

A balanced 250-example stress set was also generated, with exactly 50 examples in each equilibrium-count bucket from zero through four. This set probes tie-heavy cases and games with no pure equilibrium more directly. The baseline achieved 81.60%; the 5000-example checkpoint achieved 100.00%.

The most important base-model failure mode was overprediction on games with no pure equilibrium. On the balanced stress set, the base model solved only 16 of 50 such examples, while the 5000-example checkpoint solved all 50. This suggests that the base model often expects every 2x2 game to have at least one pure equilibrium, even though the pure-equilibrium set can be empty.

Table 5: Generalization checks beyond the canonical split.

Evaluation	Base model	5000-example SFT	95% CI, SFT	Delta
Confirmation	89.20%	99.70%	99.13–99.94%	+10.50 pp
Stress	81.60%	100.00%	98.54–100.00%	+18.40 pp

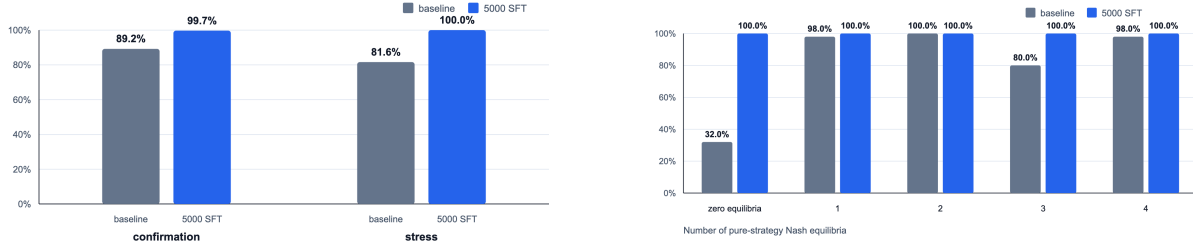


Figure 4: Confirmation and balanced-stress accuracy for the base model and original 5000-example SFT checkpoint. Left: aggregate confirmation and stress accuracy. Right: balanced-stress accuracy by number of pure equilibria, where the largest base-model weakness is the no-pure-equilibrium bucket.

10 Prompt Robustness

A 250-example robustness set tested five semantically equivalent prompt surfaces: the original table, compact payoff pairs, JSON-like payoff objects, a minimal matrix, and an answer-only instruction. The base model achieved 64.00% exact-match accuracy (95% CI: 57.71–69.95%), while the original 5000-example checkpoint achieved 88.40% (95% CI: 83.77–92.09%). The fine-tuned checkpoint therefore improved robustness substantially, but `compact_pairs` and `json_payoffs` remained weak at 78.00% and 68.00%, respectively.

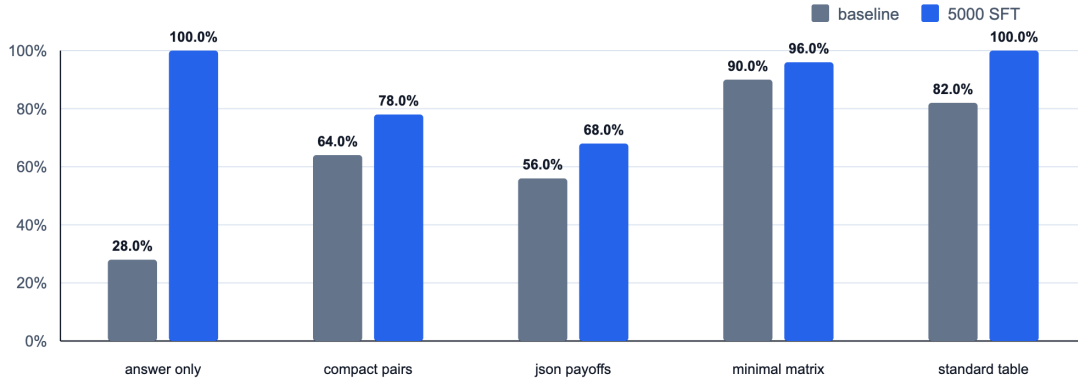


Figure 5: Prompt-robustness accuracy by prompt variant for the base model and original 5000-example SFT checkpoint.

The robustness result strengthens the main finding while also showing its boundary. The original fine-tuned checkpoint learned more than the canonical prompt template, but it was not fully invariant to equivalent payoff presentations.

11 Adversarial Prompt Fine-Tuning

The prompt-robustness follow-up added a conservative 500-example adversarial prompt supplement to the original 5000-example training set. The supplement emphasized compact payoff pairs and JSON-like payoff objects while balancing examples across the five equilibrium-count buckets. No new mathematical task types were introduced.

Table 6: Adversarial prompt supplement mix. Each variant is balanced across 0-, 1-, 2-, 3-, and 4-equilibrium games.

Prompt variant	Supplement examples	Examples per eq. count
<code>compact_pairs</code>	150	30
<code>json_payoffs</code>	150	30
<code>standard_table</code>	100	20
<code>minimal_matrix</code>	50	10
<code>answer_only</code>	50	10

Table 7 shows that adversarial supplementation raised robustness from 88.40% to 98.80% and canonical accuracy from 99.60% to 99.80%.

Table 7: Adversarial prompt SFT results.

Evaluation	Original 5000 SFT	Adversarial SFT	95% CI, adv.	Delta
Canonical	99.60%	99.80%	98.89–99.99%	+0.20 pp
Confirmation	99.70%	99.90%	99.44–100.00%	+0.20 pp
Stress	100.00%	100.00%	98.54–100.00%	+0.00 pp
Robustness	88.40%	98.80%	96.53–99.75%	+10.40 pp

Table 8: Prompt-variant robustness after adversarial SFT.

Prompt variant	Original 5000 SFT	Adversarial SFT	Delta
<code>answer_only</code>	100.00%	100.00%	+0.00 pp
<code>compact_pairs</code>	78.00%	100.00%	+22.00 pp
<code>json_payoffs</code>	68.00%	94.00%	+26.00 pp
<code>minimal_matrix</code>	96.00%	100.00%	+4.00 pp
<code>standard_table</code>	100.00%	100.00%	+0.00 pp

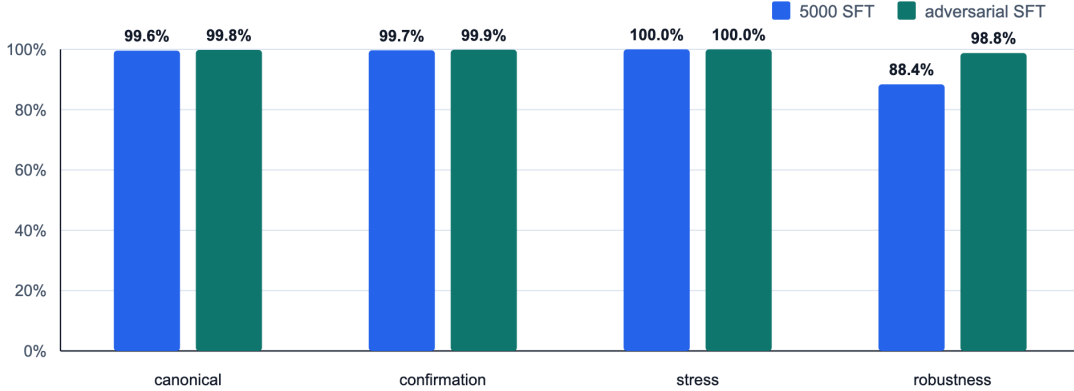


Figure 6: Original 5000-example SFT versus adversarial prompt SFT across the four evaluation sets.

12 Retention-Aware Multitask Sweep

The suite-only checkpoint established that the broader suite was learnable, but it also exposed severe canonical forgetting. The final sweep therefore used retention-aware multitask recipes. Two recipes started from the adversarial prompt checkpoint and continued training on suite, canonical-retention, and adversarial examples. Two recipes started from the base model and mixed canonical, adversarial, suite, and targeted suite examples from the beginning. The targeted suite augmentation was train-only and deterministic, using seed 20260512 with extra examples for the weakest suite families.

Selection used a retention gate: require at least 99.0% canonical accuracy and at least 95.0% prompt-robustness accuracy, then maximize broader-suite accuracy. Table 9 shows the four candidate checkpoints. The selected run was `joint_adv_targeted_retention`: it reached 91.67% suite accuracy, 99.80% canonical accuracy, 100.00% confirmation accuracy, 100.00% balanced-stress accuracy, and 99.60% prompt-robustness accuracy. The highest-suite candidate reached 93.00%, but it missed the retention gate with 97.20% canonical and 94.40% robustness accuracy.

Table 9: Retention-aware multitask sweep. The selected checkpoint maximizes suite accuracy among candidates satisfying the canonical and robustness gates.

Recipe	Canonical	Robustness	Suite	Selection
<code>joint_base_canon_adv_suite</code>	99.80%	99.60%	84.67%	eligible
<code>joint_adv_suite_retention</code>	100.00%	100.00%	89.33%	eligible
<code>joint_adv_targeted_retention</code>	99.80%	99.60%	91.67%	selected
<code>joint_base_full_targeted</code>	97.20%	94.40%	93.00%	missed gate

The selected recipe was repeated for seeds 42, 1009, and 2027. Seed 42 is the selected checkpoint above. Seeds 1009 and 2027 used seed-specific canonical-retention rows while keeping the fixed public suite split unchanged. Table 10 reports the result. Suite accuracy averaged 92.56% with 0.63 percentage-point seed standard deviation; canonical accuracy averaged 99.93% with 0.09 percentage-point seed standard deviation. The largest per-family seed standard deviations were 3.40 pp for larger normal-form games, 2.49 pp for mixed 2x2 games, and 1.63 pp for dominance.

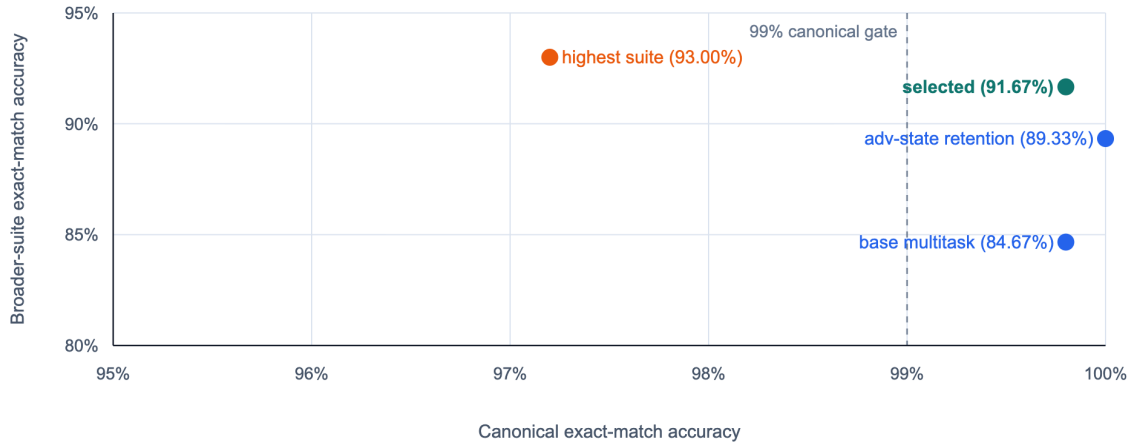


Figure 7: Canonical retention versus broader-suite accuracy for the multitask candidates. The selected point satisfies the retention rule; the highest-suite point is not selected because it falls below both retention thresholds.

Table 10: Three-seed repeat of the selected retention-aware recipe.

Seed	Canonical	Suite
42	99.80%	91.67%
1009	100.00%	93.00%
2027	100.00%	93.00%
Mean	99.93%	92.56%
Seed SD	0.09 pp	0.63 pp

Two additional base-model comparisons were chosen from the Tinker availability report. The preferred Qwen/Qwen3.6-8B and Qwen/Qwen3.6-72B models were unavailable, so the nearest available Qwen-family models below and above 27B were used. These are base-only comparisons, not fine-tuned models. Under the exact output parsers, both were weak on the canonical and suite tasks.

Table 11: External base-model comparisons selected by model availability.

Base model	Canonical	Suite
Qwen/Qwen3-8B	2.60%	4.00%
Qwen/Qwen3-30B-A3B	3.80%	6.33%

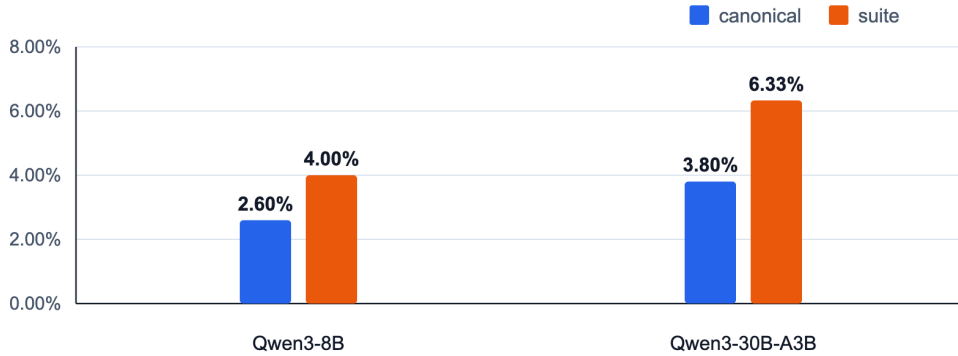


Figure 8: Base-only canonical and suite exact-match accuracy for the availability-selected external model comparisons.

13 Failure Modes

Before fine-tuning, the dominant failure mode was false positives on games with no pure equilibrium. The model frequently selected at least one cell even when no cell was a mutual best response. After 5000-example fine-tuning and adversarial prompt fine-tuning, this failure mode was mostly removed. The selected joint checkpoint has one canonical error: a tie-heavy game with three equilibria where the model misses one true equilibrium. On the broader suite, its 25 failures are concentrated in dominance (16), larger normal-form games (7), and mixed 2x2 games (2).

Table 12: Selected joint-checkpoint canonical failure example `example_005555`. Gold equilibria are (U, L) , (U, R) , and (D, L) . The checkpoint predicted only (U, L) and (U, R) , missing the tie-induced equilibrium (D, L) .

	L	R
U	(7, 4)	(4, 4)
D	(7, 9)	(3, 9)

The robustness evaluation exposed a second failure mode: compact or structured payoff presentations can reduce accuracy even when the underlying mathematical task is unchanged. The adversarial prompt supplement substantially reduced this weakness, and the selected joint

checkpoint retained 99.60% prompt-robustness accuracy. The broader suite makes the remaining errors more specific: dominance and larger normal-form reasoning are the hardest task families for the selected checkpoint.

14 Code, Data Availability, and Ethics

The code is available at <https://github.com/setrf/gt-bench> under the MIT License. The repository tracks the canonical generator, solver, scorer, broader suite generator/scorer/baselines, Tinker scripts, public summaries, figures, and this paper source. Generated JSONL data, raw predictions, raw scorer reports, and Tinker run manifests are ignored by default because they are reproducible and can contain private sampler paths. Public summary JSON files, SVG figures, and small sample JSONL files are tracked.

This work uses synthetic payoff matrices and does not involve human subjects or personal data. The main risk is interpretive: the result could be overstated as evidence of broad game-theory ability. The headline model-result claim supports 2x2 pure-equilibrium identification under the tested Tinker setup, plus a bounded broader-suite result for the selected retention-aware checkpoint. Reproduction also depends on access to the same or comparable Tinker model, renderer, and pricing environment.

15 Limitations

The reported GT-Bench experiment is not a general game-theory evaluation. It uses synthetic data and exact answer formats, and its canonical result covers only 2x2 pure-strategy equilibrium identification with integer payoffs.

The broader suite adds exactly scored coverage for mixed strategies, dominance, larger normal-form games, sequential reasoning, natural-language descriptions, and repeated interaction, and the selected joint checkpoint performs strongly on that suite. However, the suite still omits many economically important settings, including auctions, bargaining, public goods, mechanism design, signaling games, Bayesian games, stochastic games, and open-ended strategic interaction. The result also does not prove that the model acquired a human-like concept of Nash equilibrium.

The benchmark is useful precisely because it is constrained. The rule is exact, the data is reproducible, and failures can be inspected mechanically. The appropriate claim is narrow: targeted fine-tuning can improve this specific formal reasoning behavior under the tested conditions.

16 Conclusion

GT-Bench demonstrates that a compact, reproducible Tinker fine-tuning artifact can produce measurable gains on fully verifiable formal reasoning tasks. The strongest non-adversarial checkpoint improved canonical accuracy from 87.60% to 99.60%, the three-seed sweep showed that the 5000-example condition was stable, and an adversarial prompt supplement improved robustness from 88.40% to 98.80% relative to the original 5000-example checkpoint. The retention-aware multitask sweep then produced a selected checkpoint with 91.67% broader-suite accuracy while preserving 99.80% canonical accuracy. The conclusion is deliberately bounded: targeted fine-tuning can improve a specific formal skill, targeted data can close prompt-format gaps, and retention-aware multitask data can broaden coverage without erasing the original skill under this tested setup.

A Reproducibility Appendix

The public artifact records full reproduction commands and hashes in `REPRODUCIBILITY.md`. Table 13 summarizes the local workflow used to regenerate the public data, reports, and figures.

Table 13: Compact local reproduction workflow. The repository documentation gives the complete commands, output paths, and hashes.

Stage	Local command summary
Environment	<code>python3 -m venv .venv</code> <code>.venv/bin/python -m pip install -r requirements.txt</code>
Canonical data	<code>generate_dataset.py --train 5000 --val 500 --test 500 --seed 42</code> <code>make_sweep_splits.py --config bench_config.json</code>
Stress and robustness sets	<code>generate_stress_set.py --per-count 50 --seed 314159</code> <code>generate_robustness_set.py --per-bucket 10 --seed 271828</code> <code>generate_adversarial_training.py</code>
Broader suite	<code>generate_benchmark_suite.py --train-per-family 200 --val-per-family 10 --test-per-family 50 --seed 20260511</code> <code>run_suite_baselines.py --gold data/suite/test.jsonl --train data/suite/train.jsonl</code>
Public summaries	<code>make_multitask_training.py</code> <code>select_tinker_models.py</code> <code>summarize_multitask_results.py</code> <code>make check</code>

The broader suite can be scored separately with `score_suite.py` using the public suite gold file and a JSONL prediction file as input. Dataset hashes are recorded in `REPRODUCIBILITY.md` and the public summary JSON files.

The Tinker environment used package versions `tinker==0.18.2` and `tinker-cookbook==0.3.0`. The model was `Qwen/Qwen3.6-27B`, the renderer was `qwen3_disable_thinking`, and the main LoRA settings were rank 16, 3 epochs, effective batch size 32, learning rate 10^{-4} , temperature 0, and one sample per example. Retention-aware continuation runs use the manifest `state_path` with `run_tinker_sft.py --load-state-path`; long runs can be chunked with `--start-step` and `--max-steps` without changing the deterministic training file.

B Scoring Details

The parser first prefers text after the last **Final answer:** or **Conclusion:** marker, then searches answer-like sentences, then falls back to the whole output. If an empty-set phrase appears before any profile mention in the candidate span, the prediction is parsed as the empty set. Otherwise, all profile mentions matching U or D paired with L or R are extracted into a set. Order and duplicate mentions do not matter. If no prediction can be parsed, the example is scored incorrect.

The test suite covers solver behavior for one-equilibrium, two-equilibrium, no-pure-equilibrium, and tie-heavy games, as well as common parser formats. The scorer intentionally remains simple so failures are inspectable.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021. <https://arxiv.org/abs/2107.03374>.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021. <https://arxiv.org/abs/2110.14168>.
- [3] Jinhao Duan, Renming Zhang, James Diffenderfer, Bhavya Kailkhura, Lichao Sun, Elias Stengel-Eskin, Mohit Bansal, Tianlong Chen, and Kaidi Xu. GTBench: Uncovering the Strategic Reasoning Limitations of LLMs via Game-Theoretic Evaluations. *arXiv preprint arXiv:2402.12348*, 2024. <https://arxiv.org/abs/2402.12348>.
- [4] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset. In *Advances in Neural Information Processing Systems*, 2021. <https://arxiv.org/abs/2103.03874>.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. <https://arxiv.org/abs/2106.09685>.
- [6] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv preprint arXiv:2403.07974*, 2024. <https://arxiv.org/abs/2403.07974>.
- [7] John F. Nash. Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1):48–49, 1950.
- [8] John F. Nash. Non-cooperative games. *Annals of Mathematics*, 54(2):286–295, 1951.
- [9] Qwen Team. Qwen/Qwen3.6-27B Model Card. Hugging Face, 2026. <https://huggingface.co/Qwen/Qwen3.6-27B>.
- [10] Haoran Sun, Yusen Wu, Peng Wang, Wei Chen, Yukun Cheng, Xiaotie Deng, and Xu Chu. Game Theory Meets Large Language Models: A Systematic Survey with Taxonomy and New Frontiers. *arXiv preprint arXiv:2502.09053*, 2025. <https://arxiv.org/abs/2502.09053>.
- [11] Thinking Machines Lab. Announcing Tinker. 2025. <https://thinkingmachines.ai/news/announcing-tinker/>.
- [12] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025. <https://arxiv.org/abs/2505.09388>.
- [13] Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav Raja, Charlotte Zhuang, Dylan Slack, Qin Lyu, Sean Hendryx, Russell Kaplan, Michele Lunati, and Summer Yue. A Careful Examination of Large Language Model Performance on Grade School Arithmetic. In *Advances in Neural Information Processing Systems*, 2024. <https://arxiv.org/abs/2405.00332>.